GitHub URL and project name

https://github.com/KarlG322/genAI_image_project/

Patent Figure Generation

Overview and brief description of code

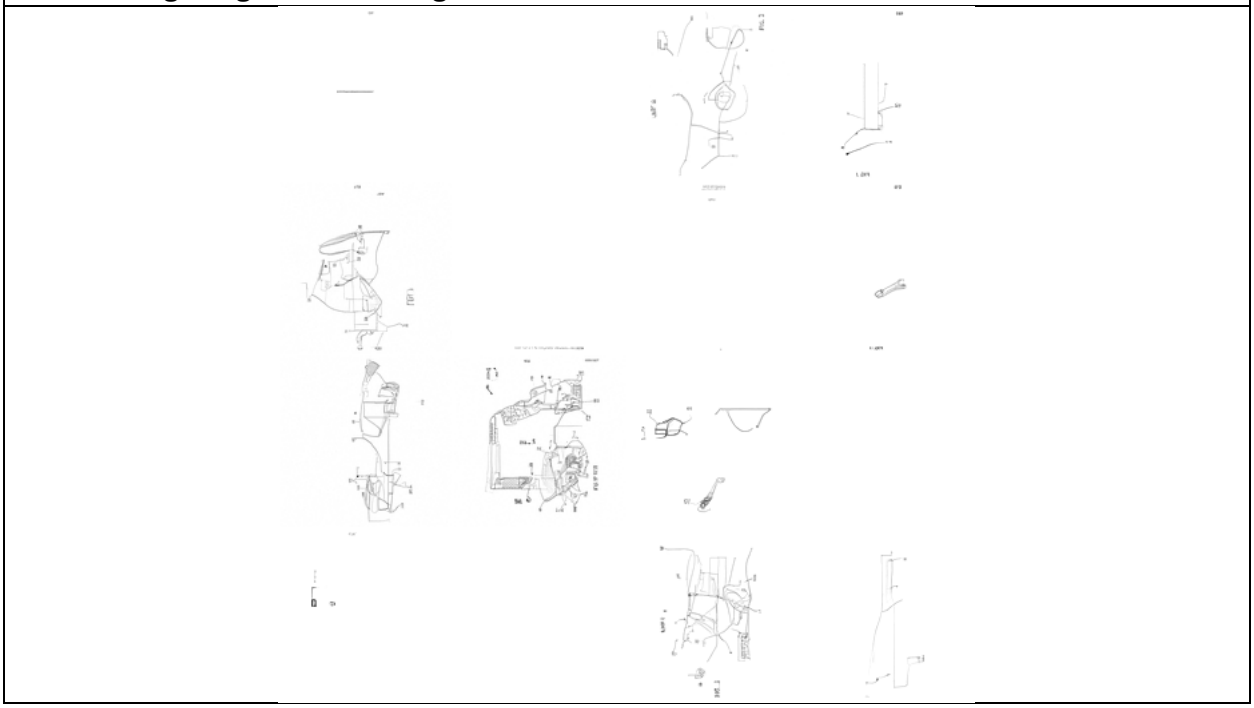
The goal was to use a diffusion model to create images that resemble patent figures. I did this by:

- Downloading PDFs of patent drawing pages from the patent office's open data portal API, manually removing the bad data, and converting them to PNGs. Note that this requires a not easily obtainable API key.
    - I made 3 datasets: first, a 2948 image 128x128 dataset (which resulted in a passable model), then a 9953 image 128x128 dataset (which resulted in a quite good model), then a 9953 image 256x256 dataset (which resulted in a poor model).
- Training a basic unet using those PNGs and using the unet to generate patent figure-like images. Brief description of the unet when the input is 128x128 (see comments in the code for a more thorough explanation):
    - 3 down blocks and a middle block compress the original [batch, 1, 128, 128] input to [batch, 256, 16, 16], then the other half of the U takes it back to [batch, 1, 128, 128]. If the inputs are 256x256, the last 2 dimensions double.
    - I use a linear diffusion schedule with 1000 steps for my 128x128 models and a no-offset cosine diffusion schedule with 1000 steps for my 256x256 model.
    - Because my images are so small, the model and data all fit in GPU memory on a Google colab L4 GPU.
- Generating images based on the trained model, and, for the extra criteria, generating images from manually created starting points that are only partially noise to guide generation (for the "latent space exploration" extra criteria). See below.

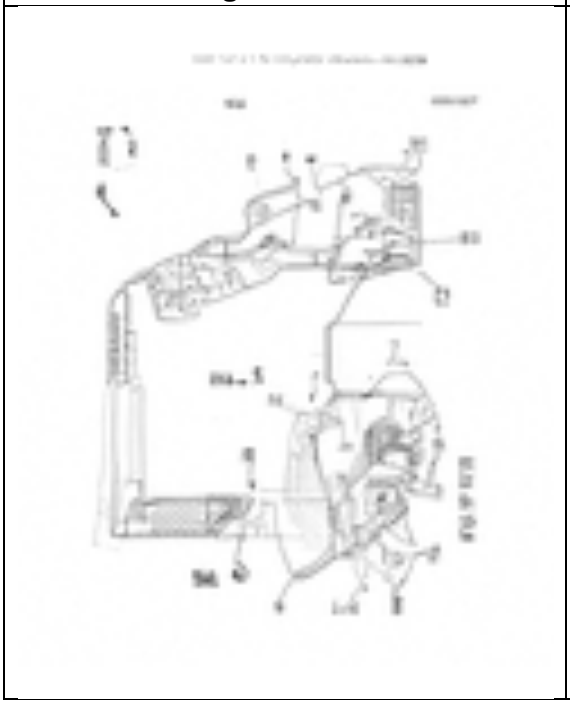
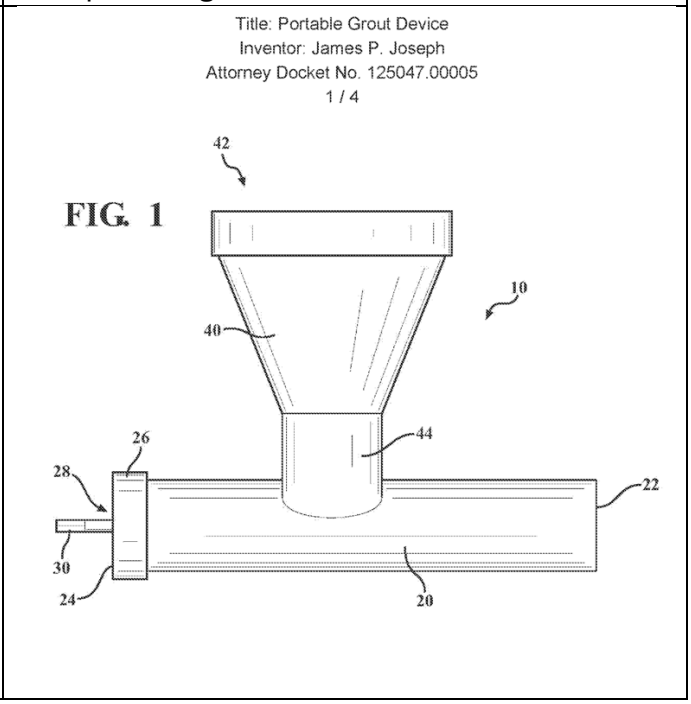
The biggest problems I had were getting data out of the USPTO API, long training times and poor performance on the 256x256 model. Of these, I only solved the API issue (by aggressively limiting what each request would pull because seemingly similar requests would pull wildly different amounts of data, sometimes so much as to generate an error). I didn't really solve the other 2 issues: I just let it run overnight and focused on my 128x128 model after the 256x256 one failed.

Results

Images generated based on the 2948 image 128x128 dataset

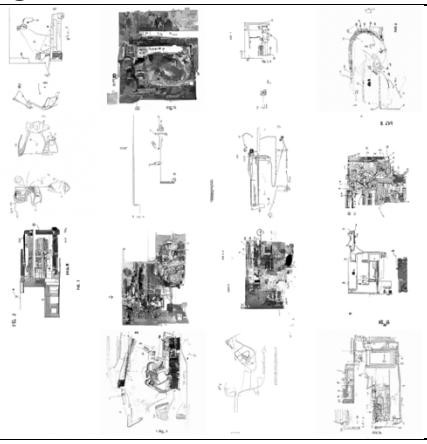
4x4 montage of generated images


- At larger sample sizes, this pattern of a few decent images with many poor ones is consistent.

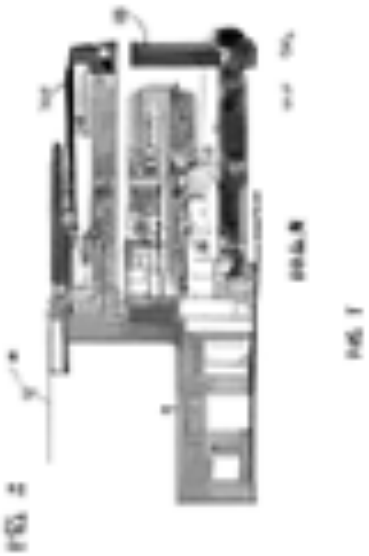
Generated image	Real patent figure for reference
	

- In the generated image, note the heading at the top of the page, the figure labels throughout, and the realistic looking joint and shading in the bottom left.
- That said, I'm not sure what the top and right parts are supposed to be.

Images generated based on the 9953 image 128x128 dataset

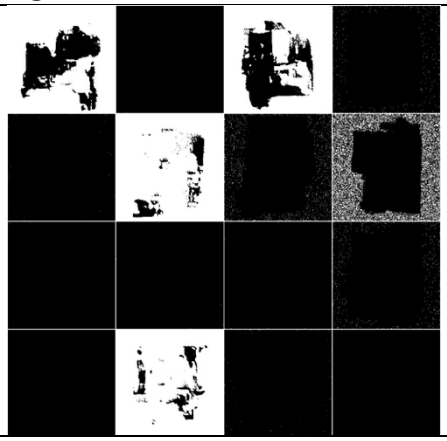
4x4 montage of generated images


- Note that the images are on average much better than with the smaller dataset.



- This one might even pass for a low-res cross section of a machine.
- While some of these look decent on the surface, the model has not learned how to make things that would make sense mechanically.

Images generated based on the 9953 image 256x256 dataset

4x4 montage of generated images


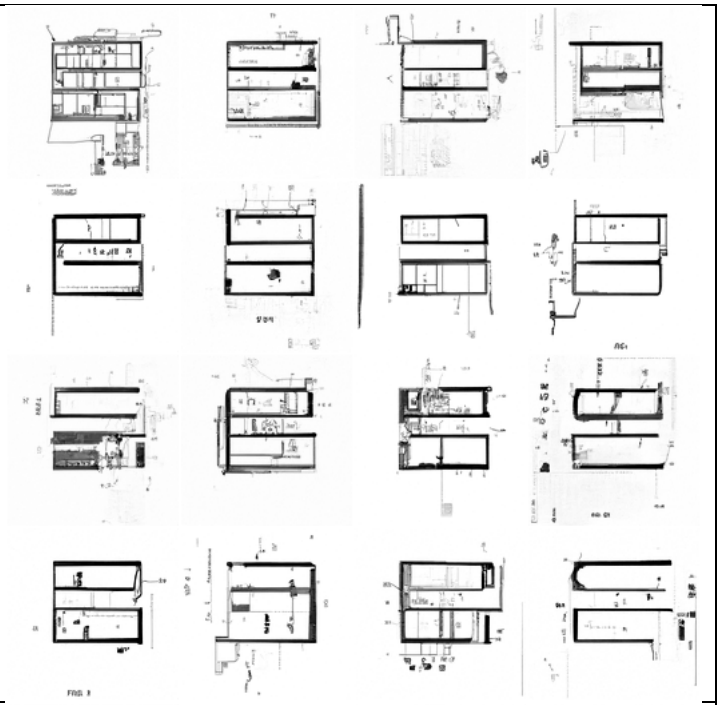
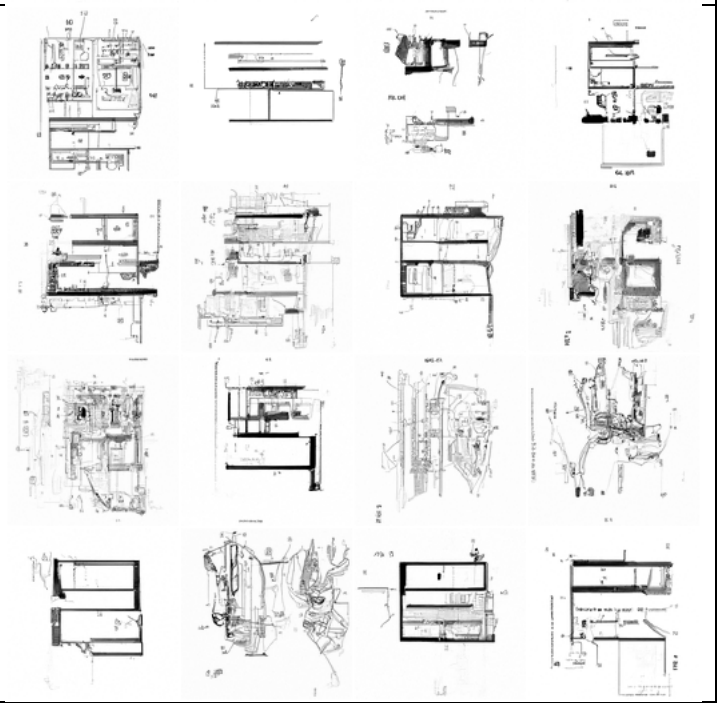
- While I had set out to make 128x128 images, I tried the model on 256x256 as well (the code works for different sizes of images, though the dimensions of each step of the unet changes if the image size changes). It clearly did not work. My guess is that I would want a slightly deeper unet and more training data if I want to make 256x256 work well.

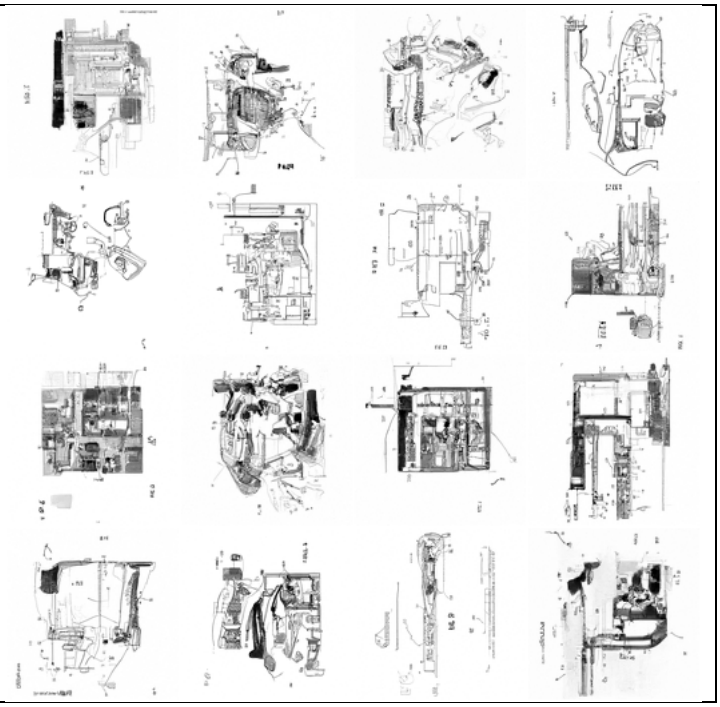
Extra criteria

I generated images based on starting points by:

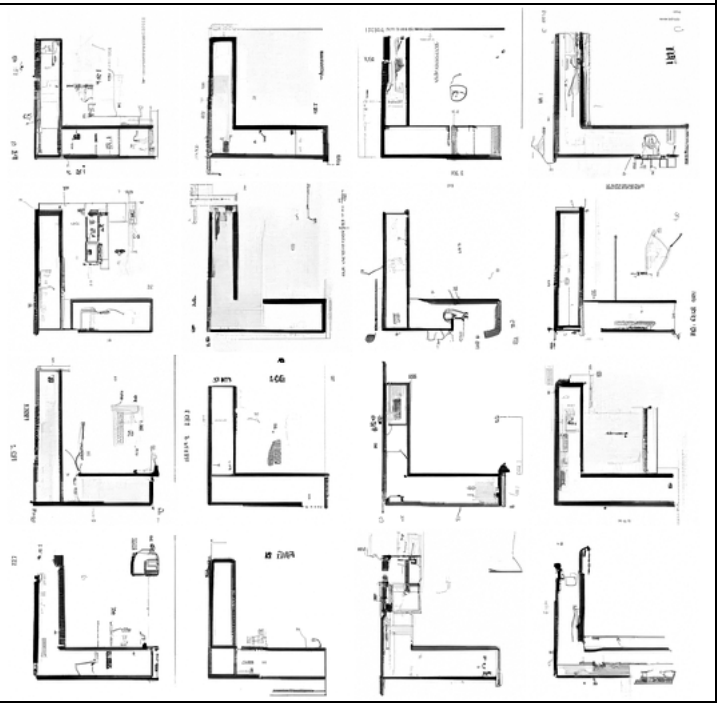
- Manually making a starting point image, and
- Using a modified version of my image generation code to start with that image, and part of the way through the diffusion process. How far through the process it started would determine how similar the outputs are to the input image (starting_noise goes from 0 to 1, with higher values meaning the final generation is less correlated with the input image).

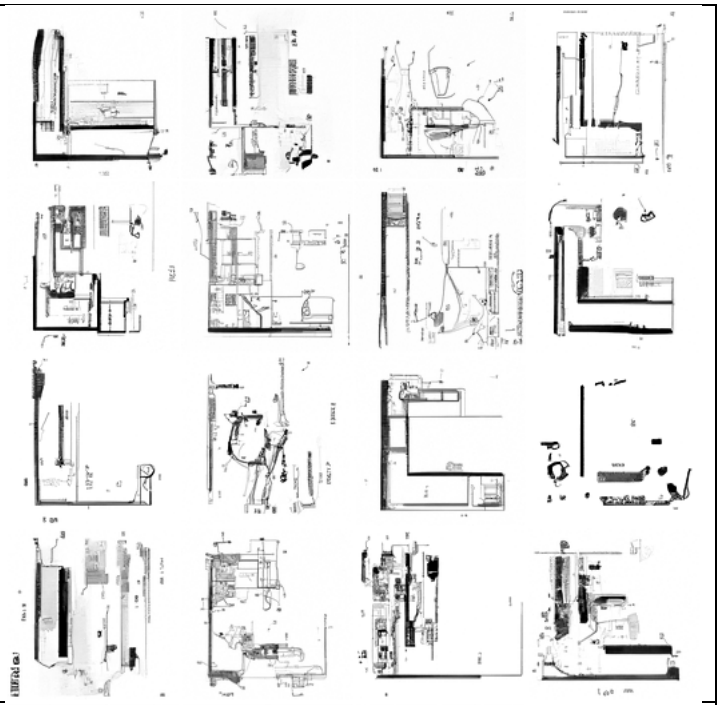
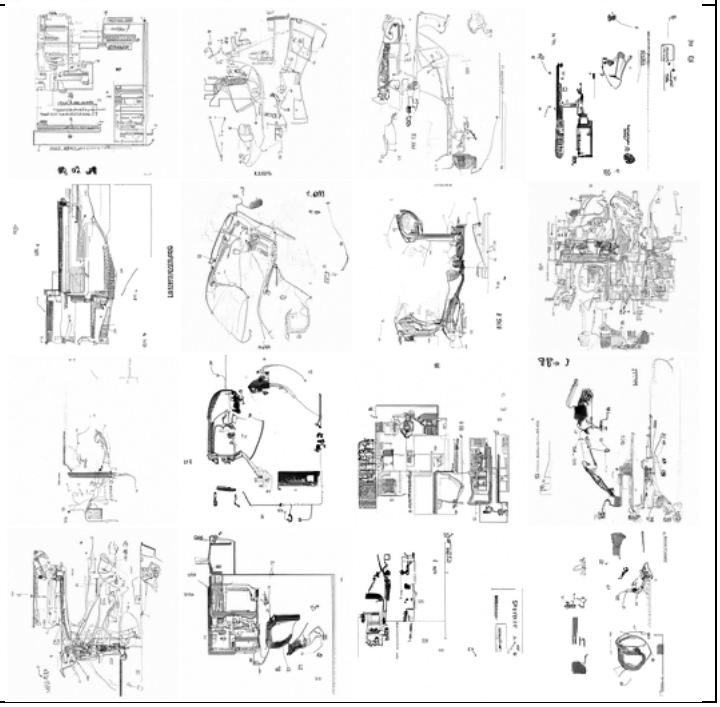
Starting point	

Generated images with starting_noise = 0.50	
Generated images with starting_noise = 0.55	

Generated images with starting_noise = 0.60	

- There's a huge difference in how much of the starting image is retained from starting_noise 0.50 to 0.60. My guess is that this is partially due to the linear diffusion schedule making some other steps learn less.

Starting point	
Generated images with starting_noise = 0.50	

Generated images with starting_noise = 0.55	
Generated images with starting_noise = 0.60	

- I find it interesting how the model's learned (at least with starting noise 0.55 or lower) to straighten out the somewhat curved lines of the input.

How to run my code

It’s designed to run on colab. I used an L4 GPU runtime.

- First, update the directories at the top of cell 2, 4 and 5 to corresponding locations in your google drive. Make sure those folders exist and there is training data in grayscale PNG format in the right folder.
- Then, run the first 2 cells (mounting drive and loading the model)
- Then, run any of the following cells:
    - Cell 3 is for continuing to train the model from the starting point you loaded (or from scratch if there's no model there)
    - Cell 4 is for generating images based on the model.
    - Cell 5 is for generating images based on the model and a manually created starting point such as an outline of a line drawing. This is for the "latent space exploration" extra criteria.